View Coherence: the Write Surface of Installed Types

Deep Dive H for "Refinement and Inference Growth in SKJS"

Models revision `cf5a6bc7` of `main` (2026-07-09)

Abstract

The main paper's membership semantics is get-only and snapshot; its Write Authorization definition (registry row **SET-SEM**) gives each view a second surface — the writes its `set`-types accept — and **L-MUT-ALIAS** states the coherence property over it: a checked write through one view preserves the membership guarantees of every live alias of the same runtime object. This deep dive audits that property site by site, over every way the checker gives a program a view: subtyping admission, refinement installation, and outright minting. The audit isolates two mechanisms that together imply coherence — the admission gate's downward write-capability preservation (D's row L-SUB-FIELDS-SET) and *write-self-consistency* of the source (`set`-types below `get`-types) — and shows the checker enforces the first while refinement destroys the second. Declaration-only aliasing is coherent: the covariant field upcast, the mutable-array upcast, and the narrowing subclass override are all rejected, each stricter than tsc. Two sites are refuted with executed witnesses: narrowing-admitted aliases (a get-narrowed object passes the gate into a narrower view, then a legal write through the original goes stale on the alias) and the minted mutable `Array`$\langle\top\rangle$ view of `instanceof Array` narrowing. The generic `instanceof` arm shares the rigid-witness defect of deep dive A's finding on the write side; the intersection's permissive `set`-union is mechanism without a found route. Fix directions accompany each verdict.

Contents

1 Purpose and Ledger Rows Addressed **2**

2 The Write Surface, Recapped **2**

3 The Coherence Property and Its Two Mechanisms **2**

4 Site Audit **3**

4.1 Declared admission: the gate holds, stricter than tsc 3

4.2 Narrowing-admitted sources: refuted, witnessed 3

4.3 Minted views: refuted at `instanceof Array`; shared-root at the generic arm 4

4.4 The intersection's `set`-union: mechanism without a found route 4

4.5 Read-path merge, accessors 5

5 Status and Ledger Edits **5**

6 Implementation Anchor Index **6**

1 Purpose and Ledger Rows Addressed

This document is deep dive H in the series supporting the main paper (*Refinement and Inference Growth in SKJS*). It owns the registry row **L-MUT-ALIAS** (cross-view mutation coherence). Every other row this paper names is an import; the authoritative list, with each row's canonical statement, owner, and status, is the generated review contract in Section 5 — citations in the body are checked against it.

The division of labor with deep dive F is by axis. F is temporal: does an installed view *stay* valid across mutation events (invalidation policy, the W-Field re-widening rule)? This paper is spatial: is the *set of simultaneously live views* of one object coherent at the moment a view is created? The two compose into the write-preservation clause of an eventual heap-level soundness statement: this paper for coherence at creation, F for preservation afterwards.

impl: `TTypeSubtyping.sk / TFields::isSub_; TypeCheckRefine.sk / narrowField, evalInstanceof; TType.sk / TFields::resolveFieldType`

2 The Write Surface, Recapped

We use the main paper's Write Authorization definition unchanged (registry row **SET-SEM**): a view b authorizes the write of v at key p iff $v \in [\![\mathsf{setty}(b,p)]\!]$, where $\mathsf{setty}(b,p)$ resolves to the intersected set-types of a named field, else the covering index's set-type, and readonly ($\mathsf{set} = \bot$) authorizes nothing. The checker realizes the definition at assignment sites by checking the right-hand side against setty with `isSub` — so every verdict below is conditional on **L-SUB** exactly where a subtype check decides it. Writes mediated by method calls (a setter-shaped method, `push`) are the province of the signature semantics **E-SIG** and are out of scope here; where a minted view widens *method parameter* types the exposure is noted but not graded.

Write $\mathsf{getty}(b,p)$ for the resolved get-type ($f.\mathsf{get}$, or the covering index's). Throughout, a *view* of a runtime object o is an object branch recorded in some scope entry (a local's flow type, a field's type) through which checked code can reach o; a view is *live* while such an entry exists.

3 The Coherence Property and Its Two Mechanisms

Definition 1 (Cross-View Coherence (registry row L-MUT-ALIAS))**.** *A set V of live views of the same runtime object is* coherent *at key p iff every write authorized by any $v_1 \in V$ preserves the membership clause of every $v_2 \in V$ at p:* $[\![\mathsf{setty}(v_1,p)]\!] \subseteq [\![\mathsf{getty}(v_2,p)]\!]$ *for all $v_1, v_2 \in V$.*

Definition 2 (Write-Self-Consistency)**.** *A view v is* write-self-consistent *at p iff* $[\![\mathsf{setty}(v,p)]\!] \subseteq [\![\mathsf{getty}(v,p)]\!]$ *— the writes it authorizes keep its own reads true. A declared data field has* $\mathsf{set} = \mathsf{get}$ *and is self-consistent by construction; a readonly field is self-consistent vacuously.*

Lemma 3 (Coherence Along Admission Edges)**.** *Let view y be admitted from view x by an accepted `isSub` (assignment, argument passing, return). If x is write-self-consistent at p, then $\{x, y\}$ is coherent at p, conditional on the accepted subtype checks (**L-SUB**) and the field-arm capability preservation (**L-SUB-FIELDS-SET**).*

Proof. Acceptance gives $[\![\mathsf{getty}(x,p)]\!] \subseteq [\![\mathsf{getty}(y,p)]\!]$ (covariant get check) and $[\![\mathsf{setty}(y,p)]\!] \subseteq [\![\mathsf{setty}(x,p)]\!]$ (contravariant set check, row L-SUB-FIELDS-SET; the readonly gate is its $\bot$ case). A write through y: $v \in [\![\mathsf{setty}(y,p)]\!] \subseteq [\![\mathsf{setty}(x,p)]\!] \subseteq [\![\mathsf{getty}(x,p)]\!]$ (self-consistency of x) $\subseteq [\![\mathsf{getty}(y,p)]\!]$ — both views' clauses hold. A write through x: $v \in [\![\mathsf{setty}(x,p)]\!] \subseteq [\![\mathsf{getty}(x,p)]\!] \subseteq$

$[\![\mathsf{getty}(y,p)]\!]$ — likewise. Coherence extends along chains of admissions from one self-consistent root by the same computation. □

Corollary 4 (Declaration-Only Aliasing is Coherent)**.** *If every view of o arises from declared object types by admission chains, coherence holds at every data-field key: declared fields are self-consistent (Definition 2), and Lemma 3 covers each edge. (Accessor fields are the separate row* ***L-GETSET****.)*

So there are exactly two ways to break coherence, and they organize every site audited below: give a program a view whose source is *not* self-consistent (Section 4.2 — refinement narrows get only, so a narrowed view has set $\not\subseteq$ get), or give it a view that never crossed the admission gate at all (Section 4.3 — minted views).

The same split re-derives deep dive F's #598 with no new machinery: a write *through the narrowed view itself* is authorized by the wide setty and lands outside the narrowed getty — self-inconsistency exploited through one view; F's W-Field rule re-widens the written view's get and restores its self-consistency at the write. The residue this paper grades is the two-view case, which W-Field on the written view cannot mend.

4 Site Audit

4.1 Declared admission: the gate holds, stricter than tsc

Three control probes against the pinned checker (`target/host/dev/skjs`, via `make check-file-in-docker`) confirm Corollary 4's premises are enforced:

- *Covariant field upcast, rejected.* {`f: number`} into {`f: number | string`}: the contravariant set check fails ("Type string ... is not assignable to type number"). tsc accepts this (property covariance); SKJS is stricter, trading incompleteness for the coherent pair.
- *Mutable-array upcast, rejected.* `Array<number>` into `Array<unknown>` fails the contravariant parameter check on methods. tsc accepts (method bivariance).
- *Narrowing subclass override, rejected.* `class E extends D` redeclaring `f: number` over `f: number | string` is rejected ("incorrectly extends its base class"). tsc accepts; the override would mint an incoherent pair at every E-to-D upcast.

Verdict: for declaration-derived views, the admission gate discharges its half of coherence — conditional on **L-SUB**'s open strata, exactly as row L-SUB-FIELDS-SET records.

impl: `TTypeSubtyping.sk / TFields::isSub_ (shared/left/right arms); TypeCheckClass.sk (override checking)`

4.2 Narrowing-admitted sources: refuted, witnessed

Field narrowing updates the get-type only; the set-type keeps its declared width (the same fact deep dive F pins for #598). The narrowed view is therefore not write-self-consistent, and Lemma 3's hypothesis fails exactly where the gate cannot see it: admission checks the covariant get against the *narrowed* get and the contravariant set against the *declared* set, so both pass.

Finding 5 (A get-narrowed source is admitted; a legal write through it goes stale on the alias)**.** *A get-narrowed object is admitted into a strictly narrower view, and a write legal for the source's declared* set*-type falsifies the alias's get clause — the failure needs only the source to be writable,*

not the alias (a readonly target is admitted and goes stale the same way). Executed witness, readonly variant, and tsc comparison: issue draft `issues/narrowed-alias-stale-write.md`. The admission-time repair restores Lemma 3's hypothesis exactly where it fails: when a source field's get-type sits strictly below its set-type, additionally require the source's set below the target's get — declared fields are unaffected.

impl: `TypeCheckRefine.sk / narrowField (withGet); TTypeSubtyping.sk / TFields::isSub_ (shared arm); TType.sk / TFields::resolveFieldType (isLValue)`

4.3 Minted views: refuted at `instanceof Array`; shared-root at the generic arm

A *minted* view is installed by a refinement rule without any `isSub` admission, so Lemma 3 does not apply at all; coherence is the minting rule's own obligation.

`instanceof Array` (refuted, witnessed). `evalInstanceof` special-cases `Array` to the closed argument $\top$ on the *mutable* `Array` class, so the minted view's index set-type is $\top$, above any actual element type: the pair it forms with the scrutinee's other views is incoherent by construction. Executed witness and repair: issue draft `issues/instanceof-array-mutable-top-view.md`; the repair recorded there (`ReadonlyArray`, as the `Array.isArray` path already does) is the vacuous case of Definition 2 — no write surface, self-consistent for free.

impl: `TypeCheckRefine.sk / evalInstanceof (Array arm), evalIsArray (ReadonlyArray contrast)`

Generic `instanceof` (shared root, not separately witnessed). For a generic class the minted view's field set-types mention the fresh unbound arguments of `makeTArgs`. Under canonical **TVAR-SEM** an unbound argument denotes $\emptyset$ — so the minted view authorizes no writes, vacuously coherent as installed — but a write's `isSub` check against an unbound candidate can *bind* it to the right-hand side's type, manufacturing write authorization from the program context the same way covariant reads manufacture read guarantees. This is the representation defect of issue draft `issues/instanceof-type-argument-sealing.md` surfacing on the write side; the rigid-witness repair there covers both sides, and this paper adds no separate witness.

impl: `TEnv.sk / makeTArgs; TTypeSubst.sk / Subst::grow`

Other refinement rules. R-Eq, R-EqField, R-IsArray, and R-In install intersections of *already-live* views (the scrutinee's current type against a value type, brand, or key fact) rather than views of new provenance; their write-surface behavior reduces to the intersection's set direction, audited next.

4.4 The intersection's set-union: mechanism without a found route

`TField::inter` *unions* the operands' set-types (deep dive B flags the direction and defers its write-surface grading here). By **L-UNION** the unioned type accepts every value either operand's set accepts — the permissive direction: a write only one operand's view authorizes passes through the merged view, and the *other* operand's get-clause has no reason to survive it. The mechanism is real. What this audit could not produce is a route to an incoherent live pair through it:

- *By declaration*: an intersection-typed target (`A & B`) has its set-types *widened* by the union, so the contravariant gate rejects admission into it from either operand's type (probed at the pin: rejected). tsc, which intersects property types instead, rejects the subsequent write; SKJS rejects the admission — different mechanisms, same outcome.

- *By refinement*: the rules of Section 4.3 install intersections of the scrutinee's own views; producing a runtime value that inhabits two views with genuinely different set-types at one key requires an admission or a mint that Sections 4.1 and 4.2–4.3 already grade.

Verdict: no independent defect established; the direction stays guarded by the admission gate on every route found. Recorded as the open residue of L-MUT-ALIAS — a sound repair for Finding 5 should keep it in view, since narrowing-installed intersections are exactly where the union becomes reachable without a gate.

impl: `TTypeInter.sk / TField::inter (set => TType::union)`

4.5 Read-path merge, accessors

The multi-branch read merge (`TFields::union`) *intersects* set-types — the conservative dual, writes through the summary authorized only if every branch authorizes them — so the branch-merge client adds no exposure (deep dive B proves the read direction; the set direction is restrictive by construction). Accessor fields, where get and set are distinct declared members and self-consistency is not a syntactic identity, are the open row **L-GETSET** and are not graded here.

impl: `TTypeUnion.sk / TField::union (set => inter)`

5 Status and Ledger Edits

Verdict for L-MUT-ALIAS. The row's status is rendered from the registry below; the evidence this paper contributes decomposes site by site — two witnessed refutations, one shared-root defect, one residue:

- narrowing-admitted aliases: refuted, witnessed (Finding 5);
- minted `instanceof Array` view: refuted, witnessed (Section 4.3);
- generic `instanceof` write-side binding: shared root with the rigid-witness representation defect, no separate witness (Section 4.3);
- the intersection set-union: mechanism present, no reachable incoherent pair found (Section 4.4); remains a residue.

Declaration-only aliasing is coherent by Corollary 4, conditional on **L-SUB**/**L-SUB-FIELDS-SET**, with the three rejection probes of Section 4.1 pinning the gate's behavior; each refuted site carries its fix direction where it is graded.

Registry. The authoritative statuses are maintained in the registry (`docs/ledger.yaml`); this paper is cited there as evidence. The rows it supports:

Row	Statement	Status
L-MUT-ALIAS	A checked write through one mutable view preserves the membership guarantees of every live alias of the same runtime object.	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-UNION	`union` over-approximates set union (all levels).	B	PROVED
L-SUB	`Clauses::isSub` acceptance implies semantic inclusion under the current state.	D	OPEN
L-MUT	Field writes are governed by `set`-types; get-only membership is sound for snapshot reads.	main	ASSUMED
L-GETSET	Get/set coherence of accessor fields.	F	OPEN
E-SIG	Behavioral call-soundness semantics for function signatures (required for function subtyping to be a soundness statement).	D	ASSUMED
TVAR-SEM	A type-variable occurrence denotes its substitution envelope intersected with its occurrence constraints; an unbound envelope is empty.	main	PROVED
SET-SEM	A view authorizes exactly the writes its resolved `set`-types accept; a readonly field authorizes nothing.	main	PROVED
L-SUB-FIELDS-SET	`isSub`'s field arms preserve write capability downward: acceptance implies each target field's `set`-type is below the source's, and readonly fields acquire no writers.	D	OPEN

6 Implementation Anchor Index

This document	Implementation
write resolution	`TType.sk / TFields::resolveFieldType (isLValue)`; `TypeCheckExpr.sk / assign`
admission gate, field arms	`TTypeSubtyping.sk / TFields::isSub_`, `isSub_NamedFields`
override checking	`TypeCheckClass.sk`
field narrowing (get-only)	`TypeCheckRefine.sk / narrowField`, `evalFieldEqualityType (withGet)`
minted views	`TypeCheckRefine.sk / evalInstanceof (Array arm)`, `evalIsArray`; `TEnv.sk / makeTArgs`
intersection set direction	`TTypeInter.sk / TField::inter (set => TType::union)`
merge set direction	`TTypeUnion.sk / TField::union (set => inter)`
write-side candidate binding	`TTypeSubst.sk / Subst::grow`